

Giáo trình Lập trình căn bản

Lâm Hoài Bảo, Dương Văn Hiếu, Nguyễn Văn Linh

Khoa Công Nghệ Thông Tin & Truyền Thông, Đại Học Cần Thơ

2005

MỤC LỤC

Mở đầu.....	2
Chương 1: GIẢI THUẬT VÀ BIỂU DIỄN GIẢI THUẬT	3
Chương 2: TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH C	16
Chương 3: CÁC CÂU LỆNH ĐƠN TRONG C	40
Chương 4: CÁC LỆNH CÓ CẤU TRÚC	52
Chương 5: CHUỖNG TRÌNH CON (HÀM)	79
Chương 6: MẢNG	90
Chương 7: CON TRỎ	102
Chương 8: CHUỖI KÝ TỰ	117
Chương 9: KIỂU CẤU TRÚC – STRUCT	126
Chương 10: TẬP TIN	135
TÀI LIỆU THAM KHẢO	148

MỞ ĐẦU

I. MỤC ĐÍCH

Quyển sách này cung cấp cho người đọc những kiến thức cơ bản về lập trình thông qua ngôn ngữ lập trình C. Đây là cơ sở để người đọc từng bước tiếp cận với thế giới lập trình, từ đó người đọc có một nền tảng vững chắc để có thể tiếp thu hầu hết các lĩnh vực trong chuyên ngành Công Nghệ Thông Tin.

Các vấn đề chính được trình bày trong quyển sách này:

- Các khái niệm: ngôn ngữ lập trình, kiểu dữ liệu.
- Khái niệm giải thuật và biểu diễn giải thuật
- Ngôn ngữ lập trình C (sau đây gọi tắt là C):
 - ✓ Các thành phần của C.
 - ✓ Các kiểu dữ liệu trong C.
 - ✓ Các câu lệnh trong C.
 - ✓ Cách thiết kế và sử dụng các hàm trong C.
 - ✓ Một số cấu trúc dữ liệu trong C.

Chương 1

GIẢI THUẬT VÀ BIỂU DIỄN GIẢI THUẬT

Các vấn đề được trình bày trong chương này:

- Từ bài toán đến chương trình
- Giải thuật và biểu diễn giải thuật
- Một số khái niệm: kiểu dữ liệu, ngôn ngữ lập trình, chương trình dịch

Nội dung chính của chương này là giải thuật & các cách biểu diễn giải thuật. Lý do vì chương trình máy tính của một bài toán cụ thể được tạo ra từ các biểu diễn của giải thuật.

I. TỪ BÀI TOÁN ĐẾN CHƯƠNG TRÌNH

Theo [6], mọi bài toán đều có thể được diễn giải theo một sơ đồ chung: $A \rightarrow B$

Trong đó:

- A là giả thiết, là điều kiện ban đầu.
- B là kết luận, là mục tiêu cần đạt hoặc là cái phải tìm.
- $\rightarrow$ là suy luận, là giải pháp cần thực hiện để tìm B từ cái đã biết A.

Một bài toán trên máy tính cũng mang đầy đủ các tính chất của bài toán ở trên. Trong đó :

- A là thông tin vào (đầu vào - input)
- B là thông tin ra (đầu ra - output)
- $\rightarrow$ là chương trình tạo ra từ các lệnh cơ bản của máy tính cho phép tạo B từ A.

Như vậy, việc giải một bài toán trên máy tính là việc xác định các yếu tố đầu vào, đầu ra cũng như xác định cách giải quyết bài toán bằng chương trình máy tính.

Thí dụ: Giả sử có hai bình B1 và B2 đựng hai loại chất lỏng khác nhau, chẳng hạn bình B1 đựng rượu, bình B2 đựng nước mắm. Làm thế nào để hoán đổi chất lỏng trong 02 bình, tức bình B1 đựng nước mắm, bình B2 đựng rượu?

- Đầu vào của bài toán là 02 bình B1, B2 với mỗi bình lần lượt chứa rượu và nước mắm.
- Đầu ra của bài toán là bình B1 chứa nước mắm, bình B2 chứa rượu.
- Một dãy các bước để thực hiện bài toán này là :
 - o Có thêm một bình thứ ba gọi là B3.
 - o *Bước 1:* Đổ rượu từ bình B1 vào bình B3.
 - o *Bước 2:* Đổ nước mắm từ bình B2 sang bình B1.
 - o *Bước 3:* Đổ rượu từ bình B3 sang B2.

Trong thí dụ trên, dãy các bước để thực hiện bài toán hoán đổi chất lỏng trong 02 bình chưa phải là 1 chương trình. Tuy nhiên chương trình máy tính thực hiện bài toán này được cài đặt với các lệnh được mô tả gần với cách mô tả trong dãy các bước trên. Dãy các bước trên còn được gọi là giải thuật để giải bài toán hoán đổi chất lỏng trong 02 bình trên.

II. GIẢI THUẬT

II.1. Khái niệm giải thuật

Giải thuật là một hệ thống chặt chẽ và rõ ràng các quy tắc nhằm xác định một dãy các thao tác trên những dữ liệu vào sao cho sau một số hữu hạn bước thực hiện các thao tác đó ta thu được kết quả của bài toán.

Thí dụ: Tìm ước số chung lớn nhất của 2 số nguyên a, b.

- Đầu vào: 2 số nguyên a, b.
- Đầu ra: ước số chung lớn nhất của a, b
- Giải thuật:
 - o *Bước 1:* Nhập vào hai số a và b.

- o *Bước 2:* So sánh 2 số a , b chọn số nhỏ nhất gán cho UCLN.
- o *Bước 3:* Nếu một trong hai số a hoặc b không chia hết cho UCLN thì thực hiện bước 4, ngược lại (cả a và b đều chia hết cho UCLN) thì thực hiện bước 5.
- o *Bước 4:* Giảm UCLN một đơn vị và quay lại bước 3
- o *Bước 5:* In UCLN - Kết thúc.

II.2 Các đặc trưng của giải thuật

- Tính kết thúc: Giải thuật phải dừng sau một số hữu hạn bước.
- Tính xác định: Các thao tác máy tính phải thực hiện được và các máy tính khác nhau thực hiện cùng một bước của cùng một giải thuật phải cho cùng một kết quả.
- Tính phổ dụng: Giải thuật phải "vét" hết các trường hợp và áp dụng cho một loạt bài toán cùng loại.
- Tính hiệu quả: Một giải thuật được đánh giá là tốt nếu nó đạt hai tiêu chuẩn sau:
 - o Thực hiện nhanh, tốn ít thời gian.
 - o Tiêu phí ít tài nguyên của máy, chẳng hạn tốn ít bộ nhớ.

Giải thuật tìm UCLN nêu trên đạt tính kết thúc bởi vì qua mỗi lần thực hiện bước 4 thì UCLN sẽ giảm đi một đơn vị cho nên trong trường hợp xấu nhất thì $\text{UCLN}=1$, giải thuật phải dừng. Các thao tác trình bày trong các bước, máy tính đều có thể thực hiện được nên nó có tính xác định. Giải thuật này cũng đạt tính phổ dụng vì nó được dùng để tìm UCLN cho hai số nguyên dương a và b bất kỳ. Tuy nhiên tính hiệu quả của giải thuật có thể chưa cao; cụ thể là thời gian chạy máy có thể còn tốn nhiều hơn một số giải thuật khác `mà chúng ta sẽ có dịp trở lại trong các chương tiếp theo.

II.3 Ngôn ngữ biểu diễn giải thuật

Để biểu diễn giải thuật, cần phải có một tập hợp các ký hiệu dùng để biểu diễn, mỗi ký hiệu biểu diễn cho một hành động nào đó. Tập hợp các ký hiệu đó lại tạo thành ngôn ngữ biểu diễn giải thuật.

II.3.1 Ngôn ngữ tự nhiên

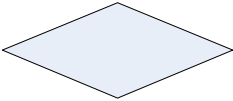
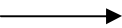
Ngôn ngữ tự nhiên là ngôn ngữ của chúng ta đang sử dụng. Chẳng hạn, các thí dụ ở trên dùng ngôn ngữ tự nhiên để biểu diễn giải thuật.

Thí dụ: Giải thuật giải phương trình bậc nhất dạng $ax + b = 0$ như sau:

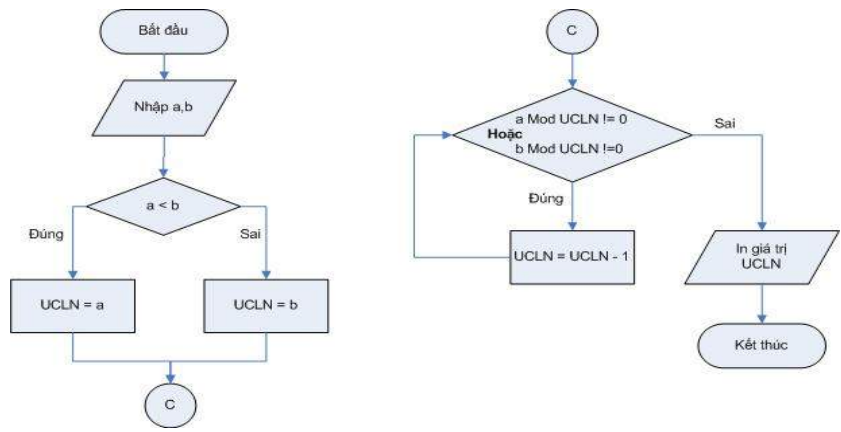
- Đầu vào: 2 số thực a, b .
- Đầu ra: Các kết luận về các trường hợp nghiệm của phương trình $ax + b = 0$.
- Giải thuật:
 - o Bước 1: Nhận giá trị của các tham số a, b
 - o Bước 2: Xét giá trị của a xem có bằng 0 hay không? Nếu $a=0$ thì làm bước 3, nếu a khác không thì làm bước 4.
 - o Bước 3: (a bằng 0) Nếu b bằng 0 thì ta kết luận phương trình vô số nghiệm, nếu b khác 0 thì ta kết luận phương trình vô nghiệm.
 - o Bước 4: (a khác 0) Kết luận phương trình có nghiệm $x=-b/a$.

II.3.2 Lưu đồ

Lưu đồ là một tập ký hiệu trực quan dùng để thể hiện giải thuật. Các ký hiệu được sử dụng trong lưu đồ được cho trong bảng sau:

Ký hiệu	Ý nghĩa
	Bắt đầu/ Kết thúc
	Nhập/Xuất
	Tính toán (xử lý)
	Quyết định và rẽ nhánh
	Khởi nối
	Đường đi

Chẳng hạn lưu đồ để biểu diễn giải thuật tìm ước số chung lớn nhất theo cách thức như trên là:



II.3.3 Một số thí dụ

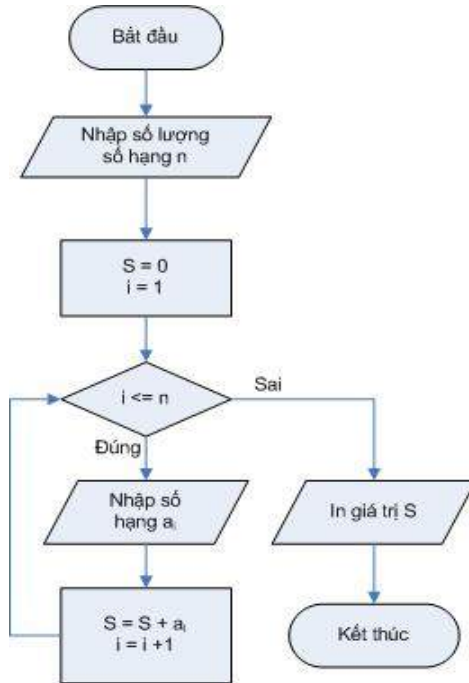
Thí dụ 1: Cần viết chương trình cho máy tính sao cho khi thực hiện chương trình đó, máy tính yêu cầu người sử dụng chương trình nhập vào các số hạng của tổng (n); nhập vào dãy các số hạng a_i của tổng. Sau đó, máy tính sẽ thực hiện việc tính tổng các số a_i này và in kết quả của tổng tính được.

Yêu cầu: Tính tổng n số $S=a_1+ a_2 + a_3 +.....+ a_n$.

Để tính tổng trên, chúng ta sử dụng phương pháp “cộng tích lũy” nghĩa là khởi đầu cho $S=0$. Sau mỗi lần nhận được một số hạng a_i từ bàn phím, ta cộng tích lũy a_i vào S (lấy giá trị được lưu trữ trong S , cộng thêm a_i và lưu trở lại vào S). Tiếp tục quá trình này đến khi ta tích lũy được a_n vào S thì ta có S là tổng các a_i . Chi tiết giải thuật được mô tả bằng ngôn ngữ tự nhiên như sau:

- o Bước 1: Nhập số các số hạng n .
- o Bước 2: Cho $S=0$ (lưu trữ số 0 trong S)
- o Bước 3: Cho $i=1$ (lưu trữ số 1 trong i)
- o Bước 4: Kiểm tra nếu $i \leq n$ thì thực hiện bước 5, ngược lại thực hiện bước 8.
- o Bước 5: Nhập a_i
- o Bước 6: Cho $S=S+a_i$ (lưu trữ giá trị $S + a_i$ trong S)
- o Bước 7: Tăng i lên 1 đơn vị và quay lại bước 4.
- o Bước 8: In S và kết thúc chương trình.

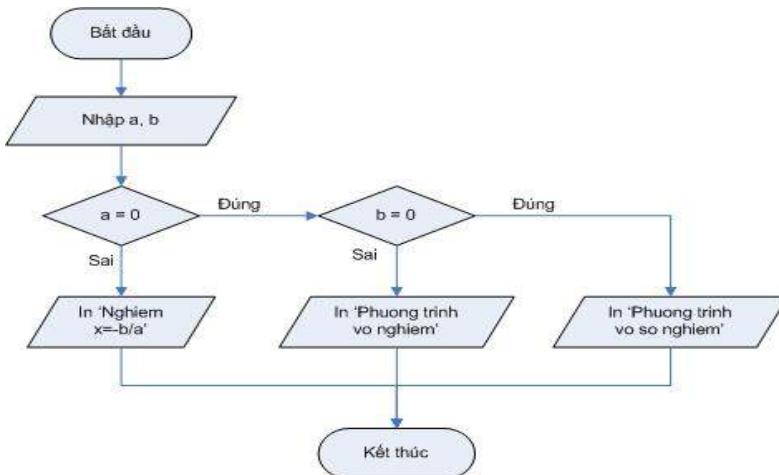
Chi tiết giải thuật trên bằng lưu đồ:



Thí dụ 2: Viết chương trình cho phép nhập vào 2 giá trị a, b mang ý nghĩa là các hệ số a, b của phương trình bậc nhất. Dựa vào các giá trị a, b đó cho biết nghiệm của phương trình bậc nhất $ax + b = 0$.

Mô tả giải thuật bằng ngôn ngữ tự nhiên (trang 6)

Lưu đồ của giải thuật giải phương trình bậc nhất:



Thí dụ 3: Viết chương trình cho phép nhập vào 1 số n , sau đó lần lượt nhập vào n giá trị $a_1, a_2, \dots, a_n$. Hãy tìm và in ra giá trị lớn nhất trong n số $a_1, a_2, \dots, a_n$.

Để giải quyết bài toán trên, chúng ta áp dụng phương pháp “thử và sửa”. Ban đầu giả sử a_1 là số lớn nhất (được lưu trong giá trị max); sau đó lần lượt xét các a_i còn lại, nếu a_i nào lớn hơn giá trị max thì lúc đó max sẽ nhận giá trị là a_i . Sau khi đã xét hết các a_i thì max chính là giá trị lớn nhất cần tìm.

Mô tả giải thuật bằng ngôn ngữ tự nhiên:

- o Bước 1: Nhập số n
- o Bước 2: Nhập số thứ nhất a_1
- o Bước 3: Gán $\text{max}=a_1$
- o Bước 4: Gán $i=2$
- o Bước 5: Nếu $i \leq n$ thì thực hiện bước 6, ngược lại thực hiện bước 9
- o Bước 6: Nhập a_i
- o Bước 7: Nếu $\text{max} < a_i$ thì gán $\text{max}=a_i$.
- o Bước 8: Tăng i lên một đơn vị và quay lại bước 5
- o Bước 9: In max - kết thúc

Phần mô tả giải thuật bằng lưu đồ, độc giả có thể xem như 1 bài tập.

Thí dụ 4: Viết chương trình cho phép nhập vào 1 số n , sau đó lần lượt nhập vào n giá trị $a_1, a_2, \dots, a_n$. Sắp theo thứ tự tăng dần một dãy n số $a_1, a_2, \dots, a_n$ nói trên.

Dưới đây là một trong những phương pháp để sắp xếp thứ tự 1 dãy các số:

Giả sử ta đã nhập vào 1 dãy n số $a_1, a_2, \dots, a_n$. Việc sắp xếp dãy số này trải qua $(n-1)$ lần:

- Lần 1: So sánh phần tử đầu tiên với tất cả các phần tử đứng sau phần tử đầu tiên. Nếu có phần tử nào nhỏ hơn phần tử đầu tiên thì đổi chỗ phần tử đầu tiên với phần tử nhỏ hơn đó. Sau lần 1, ta được phần tử đầu tiên là phần tử nhỏ nhất.

- Lần 2: So sánh phần tử thứ 2 với tất cả các phần tử đứng sau phần tử thứ 2. Nếu có phần tử nào nhỏ hơn phần tử thứ 2 thì đổi chỗ phần tử thứ 2 với phần tử nhỏ hơn đó. Sau lần 2, ta được phần tử đầu tiên và phần tử thứ 2 là đúng vị trí của nó khi sắp xếp.
- ...
- Lần (n-1): So sánh phần tử thứ (n-1) với phần tử đứng sau phần tử (n-1) là phần tử thứ n. Nếu phần tử thứ n nhỏ hơn phần tử thứ (n-1) thì đổi chỗ 2 phần tử này.

Sau lần thứ (n-1), ta được danh sách gồm n phần tử được sắp thứ tự.

Mô tả giải thuật bằng ngôn ngữ tự nhiên:

- o Bước 1: Gán $i=1$
- o Bước 2: Gán $j=i+1$
- o Bước 3: Nếu $i \leq n-1$ thì thực hiện bước 4, ngược lại thực hiện bước 8
- o Bước 4: Nếu $j \leq n$ thì thực hiện bước 5, ngược lại thì thực hiện bước 7.
- o Bước 5: Nếu $a_i > a_j$ thì hoán đổi a_i và a_j cho nhau (nếu không thì thôi).
- o Bước 6: Tăng j lên một đơn vị và quay lại bước 4
- o Bước 7: Tăng i lên một đơn vị và quay lại bước 3
- o Bước 8: In dãy số $a_1, a_2, \dots, a_n$ - Kết thúc.

II.4 Các cấu trúc suy luận cơ bản của giải thuật

Giải thuật được thiết kế theo các cấu trúc suy luận cơ bản sau đây:

II.4.1 Tuần tự (Sequential): Các công việc được thực hiện một cách tuần tự, công việc này nối tiếp công việc kia.

II.4.2 Lựa chọn (Selection): Lựa chọn một công việc để thực hiện căn cứ vào một điều kiện nào đó. Một số biến thể của cấu trúc này như sau:

- 1: Nếu < điều kiện> (đúng) thì thực hiện < công việc>
- 2: Nếu < điều kiện> (đúng) thì thực hiện < công việc 1>, ngược lại (điều kiện sai) thì thực hiện < công việc 2>
- 3: Trường hợp < i> thực hiện < công việc i>

II.4.3. Lặp (Repeating)

Thực hiện lặp lại một công việc không hoặc nhiều lần căn cứ vào một điều kiện nào đó. Có hai dạng như sau:

- *Lặp xác định*: là loại lặp mà khi mô tả giải thuật, số lần lặp của công việc đã được xác định.

- *Lặp không xác định*: là loại lặp mà khi mô tả giải thuật, số lần lặp của công việc chưa thể xác định. Tùy thuộc vào những lần thực thi khác nhau của chương trình cài đặt giải thuật, số lần lặp có thể khác nhau.

III. MỘT SỐ KHÁI NIỆM KHÁC

III.1. Ngôn ngữ lập trình

Ngôn ngữ lập trình là một ngôn ngữ dùng để viết chương trình cho máy tính. Ta có thể chia ngôn ngữ lập trình thành các loại sau: ngôn ngữ máy, hợp ngữ và ngôn ngữ cấp cao.

Ngôn ngữ máy (machine language): Là các chỉ thị dưới dạng nhị phân, can thiệp trực tiếp vào trong các mạch điện tử. Chương trình được viết bằng ngôn ngữ máy thì có thể được thực hiện ngay không cần qua bước trung gian nào. Tuy nhiên chương trình viết bằng ngôn ngữ máy dễ sai sót, cồng kềnh và khó đọc, khó hiểu vì toàn những con số 0 và 1.

Hợp ngữ (assembly language): Bao gồm tên các câu lệnh và quy tắc viết các câu lệnh đó. Tên các câu lệnh bao gồm hai phần:

phần mã lệnh (viết tựa tiếng Anh) chỉ phép toán cần thực hiện và địa chỉ chứa toán hạng của phép toán đó. Thí dụ:

INPUT a ; Nhập giá trị cho a từ bàn phím

LOAD a ; Đọc giá trị a vào thanh ghi tổng A

PRINT a; Hiện thị giá trị của a ra màn hình.

INPUT b

ADD b; Cộng giá trị của thanh ghi tổng A với giá trị b

Trong các lệnh trên thì INPUT, LOAD, PRINT, ADD là các mã lệnh còn a, b là địa chỉ. Để máy thực hiện được một chương trình viết bằng hợp ngữ thì chương trình đó phải được dịch sang ngôn ngữ máy. Công cụ thực hiện việc dịch đó được gọi là Assembler.

Ngôn ngữ cấp cao (High level language): Ra đời và phát triển nhằm phản ánh cách thức người lập trình nghĩ và làm. Rất gần với ngôn ngữ con người (Anh ngữ) nhưng chính xác như ngôn ngữ toán học. Cùng với sự phát triển của các thế hệ máy tính, ngôn ngữ lập trình cấp cao cũng được phát triển rất đa dạng và phong phú, việc lập trình cho máy tính vì thế mà cũng có nhiều khuynh hướng khác nhau: lập trình cấu trúc, lập trình hướng đối tượng, lập trình logic, lập trình hàm... Một chương trình viết bằng ngôn ngữ cấp cao được gọi là chương trình nguồn (source programs). Để máy tính "hiểu" và thực hiện được các lệnh trong chương trình nguồn thì phải có một chương trình dịch để dịch chương trình nguồn (viết bằng ngôn ngữ cấp cao) thành dạng chương trình có khả năng thực thi.

III.2 Chương trình dịch

Muốn chuyển từ chương trình nguồn sang chương trình đích phải có chương trình dịch. Nói chung các chương trình dịch hoạt động theo 1 trong 2 cơ chế:

Thông dịch (interpreter): Là cách dịch từng lệnh một, dịch tới đâu thực hiện tới đó. Chẳng hạn ngôn ngữ LISP sử dụng trình thông dịch.

Biên dịch (compiler): Dịch toàn bộ chương trình nguồn thành chương trình đích rồi sau đó mới thực hiện. Các ngôn ngữ sử dụng trình biên dịch như Pascal, C...

Giữa thông dịch và biên dịch có khác nhau ở chỗ: Do thông dịch là vừa dịch vừa thực thi chương trình còn biên dịch là dịch xong toàn bộ chương trình rồi mới thực thi nên chương trình viết bằng ngôn ngữ biên dịch thực hiện nhanh hơn chương trình viết bằng ngôn ngữ thông dịch.

Một số ngôn ngữ ở đó chương trình dịch sử dụng kết hợp giữa thông dịch và biên dịch chẳng hạn như Java. Chương trình nguồn của Java được biên dịch tạo thành một chương trình đối tượng (một dạng mã trung gian) và khi thực hiện thì từng lệnh trong chương trình đối tượng được thông dịch thành mã máy.

III.3 Kiểu dữ liệu

Các số liệu lưu trữ trong máy tính gọi là *dữ liệu* (data). Mỗi đơn vị dữ liệu thuộc một kiểu dữ liệu nào đó.

Kiểu dữ liệu là một tập hợp các giá trị có cùng một tính chất và tập hợp các phép toán thao tác trên các giá trị đó. Người ta chia kiểu dữ liệu ra làm 2 loại: kiểu dữ liệu sơ cấp và kiểu dữ liệu có cấu trúc.

III.3.1 Kiểu dữ liệu sơ cấp

Kiểu dữ liệu sơ cấp là kiểu dữ liệu mà giá trị của nó là đơn nhất.

Thí dụ: Trong ngôn ngữ lập trình C, kiểu int gọi là kiểu sơ cấp vì kiểu này bao gồm các số nguyên từ -32768 đến 32767 và các phép toán +, -, *, /, %...

III.3.2 Kiểu dữ liệu có cấu trúc

Kiểu dữ liệu có cấu trúc là kiểu dữ liệu mà các giá trị của nó là sự kết hợp của các giá trị khác.

Thí dụ: Kiểu chuỗi ký tự trong ngôn ngữ lập trình C là một kiểu dữ liệu có cấu trúc.

Các ngôn ngữ lập trình đều có những kiểu dữ liệu do ngôn ngữ xây dựng sẵn, mà ta gọi là các kiểu chuẩn. Chẳng hạn như kiểu

int, char... trong C; integer, array... trong Pascal. Ngoài ra, hầu hết các ngôn ngữ đều cung cấp cơ chế cho phép người lập trình định nghĩa kiểu của riêng mình để phục vụ cho việc viết chương trình.

IV. BÀI TẬP

Bằng ngôn ngữ tự nhiên và lưu đồ, anh (chị) hãy mô tả giải thuật cho các bài toán sau:

1. Tính diện tích của 1 tam giác theo công thức Hê-rông:

$$S = \sqrt{p*(p-a)*(p-b)*(p-c)}$$

Với a, b, c là chiều dài 3 cạnh được nhập từ bàn phím.

p: Nửa chu vi.

2. Giải phương trình bậc 2 dạng $ax^2 + bx + c = 0$ với a, b, c là các số sẽ nhập từ bàn phím.

3. Tính tổng bình phương của n số nguyên có dạng sau:

$S = a_1^2 + a_2^2 + \dots + a_n^2$ với n và a_i ($i=1..n$) là các số sẽ nhập từ bàn phím.

Chương 2

TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH C

Các vấn đề được trình bày trong chương này:

- Lịch sử của ngôn ngữ lập trình C.
- Các thành phần của C.

I. SƠ LƯỢC VỀ NGÔN NGỮ LẬP TRÌNH C

C là ngôn ngữ lập trình cấp cao, được sử dụng rất phổ biến để lập trình hệ thống cùng với Assembler và phát triển các ứng dụng.

Vào những năm cuối thập kỷ 60 đầu thập kỷ 70 của thế kỷ XX, Dennish Ritchie (làm việc tại phòng thí nghiệm Bell) đã phát triển ngôn ngữ lập trình C dựa trên ngôn ngữ BCPL (do Martin Richards đưa ra vào năm 1967) và ngôn ngữ B (do Ken Thompson phát triển từ ngôn ngữ BCPL vào năm 1970 khi viết hệ điều hành UNIX đầu tiên trên máy PDP-7) và được cài đặt lần đầu tiên trên hệ điều hành UNIX của máy DEC PDP-11.

Năm 1978, Dennish Ritchie và B.W Kernighan đã cho xuất bản quyển “Ngôn ngữ lập trình C” và được phổ biến rộng rãi đến nay.

Lúc ban đầu, C được thiết kế nhằm lập trình trong môi trường của hệ điều hành Unix nhằm mục đích hỗ trợ cho các công việc lập trình phức tạp. Nhưng về sau, với những nhu cầu phát triển ngày một tăng của công việc lập trình, C đã vượt qua khuôn khổ của phòng thí

nh nghiệm Bell và nhanh chóng hội nhập vào thế giới lập trình để rồi các công ty lập trình sử dụng một cách rộng rãi. Sau đó, các công ty sản xuất phần mềm lần lượt đưa ra các phiên bản hỗ trợ cho việc lập trình bằng ngôn ngữ C và chuẩn ANSI C cũng được khai sinh từ đó.

Ngôn ngữ lập trình C là một ngôn ngữ lập trình hệ thống rất mạnh và rất “mềm dẻo”, có một thư viện gồm rất nhiều các hàm

(function) đã được tạo sẵn. Người lập trình có thể tận dụng các hàm này để giải quyết các bài toán mà không cần phải tạo mới. Hơn thế nữa, ngôn ngữ C hỗ trợ rất nhiều phép toán nên phù hợp cho việc giải quyết các bài toán kỹ thuật có nhiều công thức phức tạp. Ngoài ra, C cũng cho phép người lập trình tự định nghĩa thêm các kiểu dữ liệu trừu tượng khác. Tuy nhiên, điều mà người mới vừa học lập trình C thường gặp “rắc rối” là “hơi khó hiểu” do sự “mềm dẻo” của C. Dù vậy, C được phổ biến khá rộng rãi và đã trở thành một công cụ lập trình khá mạnh, được sử dụng như là một ngôn ngữ lập trình chủ yếu trong việc xây dựng những phần mềm hiện nay.

Ngôn ngữ C có những đặc điểm cơ bản sau:

- *Tính cô đọng (compact)*: C chỉ có 32 từ khóa chuẩn và 40 toán tử chuẩn, nhưng hầu hết đều được biểu diễn bằng những chuỗi ký tự ngắn gọn.
- *Tính cấu trúc (structured)*: C có một tập hợp những chỉ thị của lập trình như cấu trúc lựa chọn, lặp... Từ đó các chương trình viết bằng C được tổ chức rõ ràng, dễ hiểu.
- *Tính tương thích (compatible)*: C có bộ tiền xử lý và một thư viện chuẩn vô cùng phong phú nên khi chuyển từ máy tính này sang máy tính khác các chương trình viết bằng C vẫn hoàn toàn tương thích.
- *Tính mềm dẻo (flexible)*: C là một ngôn ngữ rất uyển chuyển về cú pháp, chấp nhận nhiều cách thể hiện, có thể thu gọn kích thước của các mã lệnh làm chương trình chạy nhanh hơn.
- *Biên dịch (compile)*: C cho phép biên dịch nhiều tập tin chương trình riêng rẽ thành các tập tin đối tượng (object) và liên kết (link) các đối tượng đó lại với nhau thành một chương trình có thể thực thi được (executable) thống nhất.

II. BỘ CHỮ VIẾT TRONG C

Bộ chữ viết trong ngôn ngữ C bao gồm những ký tự, ký hiệu sau: (phân biệt chữ in hoa và in thường):

- Các ký tự hoa A, B, C...Z
- Các ký tự thường a, b, c ...z.

- Các ký số 0, 1, 2...9.
- Các ký hiệu toán học: +, -, *, /, =, <, >, (,)
- Các ký hiệu đặc biệt: ::, ;, " ' _ @ # \$! ^ [] { } ...
- Dấu cách hay khoảng trống.

III. CÁC TỪ KHÓA TRONG C

Từ khóa là các từ dành riêng (reserved words) của C mà người lập trình có thể sử dụng chúng trong chương trình tùy theo ý nghĩa của từng từ. Dưới đây là bảng liệt kê các từ khóa thông dụng của C để bạn đọc lướt qua cho có khái niệm, không nhất thiết phải nhớ ngay:

break	case	const	continue
default	do	else	for
goto	if	return	sizeof
struct	typedef	while	

IV. TÊN VÀ QUY CÁCH ĐẶT TÊN

Một chương trình nguồn C sử dụng khá nhiều tên hay còn gọi là định danh (identifier) như: tên hàm, tên biến, tên hằng... Mọi tên đều phải được khai báo trước khi sử dụng.

Tên có hai loại là tên chuẩn và tên do người lập trình đặt.

- Tên chuẩn là tên do C đặt sẵn như sin, cos, printf...
- Tên do người lập trình tự đặt để dùng trong chương trình của mình. Sử dụng bộ chữ cái, chữ số và dấu gạch dưới (_) để đặt tên, nhưng phải tuân thủ quy tắc:
 - o Bắt đầu bằng một chữ cái hoặc dấu gạch dưới.
 - o Không có khoảng trống ở giữa tên.
 - o Không được trùng với từ khóa.
 - o Không cấm việc đặt tên trùng với tên chuẩn nhưng khi đó ý nghĩa của tên chuẩn không còn giá trị nữa.

Thí dụ: Tên do hợp lệ: Chieu_dai, Chieu_Rong, Chu_Vi.

Tên không hợp lệ: Do Dai, 12A2,...

Chú ý: C là ngôn ngữ phân biệt ký tự hoa và ký tự thường, vì thế pi và Pi là 2 tên khác nhau.

V. CHÚ THÍCH

Khi viết chương trình đôi lúc ta cần phải có vài lời ghi chú về 1 đoạn chương trình nào đó để dễ nhớ và làm sáng sủa chương trình. Để ý rằng phần nội dung ghi chú này phải không thuộc về chương trình (khi biên dịch phần này bị bỏ qua). Phần ghi chú như vậy được gọi là chú thích. Trong ngôn ngữ lập trình C, nội dung chú thích phải được viết trong cặp dấu `/*` và `*/` (nếu chú thích trên nhiều dòng), hoặc đặt sau cặp dấu `//` (nếu chú thích trên 1 dòng).

Thí dụ: Chương trình hiển thị lên màn hình câu thông báo ‘Hello World’.

```
#include<stdio.h>
#include<conio.h>
main (){
    /*Xuat chuoi ra man hinh*/
    printf("Hello World");
    //Cho phim bat ky
    getch();
}
```

Ý nghĩa các dòng trong đoạn chương trình trên:

- Các dòng

```
#include<stdio.h>
#include<conio.h>
```

cho phép gộp các tập tin tiêu đề `stdio.h` và `conio.h` vào tập tin chương trình nguồn. Tập tin `stdio.h` chứa định nghĩa về các hàm xuất nhập chuẩn (chẳng hạn `printf`). Tập tin `conio.h` chứa các hàm xuất nhập như `getch()`.

`#include` là chỉ thị hướng dẫn dịch của C, chỉ thị này thông báo cho chương trình dịch nạp tập tin tiêu đề `stdio.h` (và `conio.h`) vào để dịch.

- Khối

```
main()
{
    ...
}
```

gọi là thân chương trình. Phần này bắt buộc phải có cho mọi chương trình C. Phần này còn được gọi là hàm main() hoặc là điểm bắt đầu thực hiện của 1 chương trình C.

- Dòng `printf("Hello world");` cho phép hiển thị giá trị của câu *Hello World* lên màn hình.

- Dòng `getch();` chờ nhận 1 ký tự từ bàn phím. Chương trình sẽ thực hiện tiếp tục sau khi một phím bất kỳ được ấn.

Phần nằm trong `/* */` hoặc dòng nằm sau `//` là chú thích

VI. CÁC KIỂU DỮ LIỆU SƠ CẤP CHUẨN TRONG C

Các kiểu dữ liệu sơ cấp chuẩn trong C có thể được chia làm 2 dạng : kiểu số nguyên, kiểu số thực. Phần này chỉ trình bày miền giá trị của các kiểu, các phép toán trên các giá trị của các kiểu sẽ được trình bày ở các phần tiếp theo.

VI.1. Kiểu số nguyên

Kiểu số nguyên là một kiểu dữ liệu mà tập các giá trị của nó là tập con của tập số nguyên trong toán học. Tùy thuộc vào kích thước lưu trữ mà kiểu số nguyên được chia làm 3 loại: 1 byte, 2 bytes hay 4 bytes. Ứng với mỗi loại, người ta còn định nghĩa sẵn một số kiểu khác nhau. Tùy thuộc vào ứng dụng của chương trình mà người lập trình sẽ sử dụng 1 kiểu thích hợp.

VI.1.1. Kiểu số nguyên 1 byte (8 bits)

STT	Kiểu dữ liệu	Miền giá trị (Domain)
1	char	Từ -128 đến 127
2	unsigned char	Từ 0 đến 255

Kiểu số nguyên một byte gồm có 2 kiểu sau:

- **Kiểu char:** các số nguyên thuộc kiểu **char** có giá trị từ -128 đến 127.

- **Kiểu unsigned char:** các số nguyên thuộc kiểu **unsigned char** có giá trị từ 0 đến 255.

Chú ý: Kiểu **char** còn có thể được gọi là kiểu ký tự vì C cho phép kiểu này được sử dụng như số nguyên và ký tự.

VI.1.2. Kiểu số nguyên 2 bytes (16 bits)

Kiểu số nguyên 2 bytes gồm có 4 kiểu sau:

STT	Kiểu dữ liệu	Miền giá trị (Domain)
1	int	Từ -32,768 đến 32,767
2	short	Từ -32,768 đến 32,767
3	unsigned int	Từ 0 đến 65,535
4	unsigned short	Từ 0 đến 65,535

VI.1.3. Kiểu số nguyên 4 byte (32 bits)

Kiểu số nguyên 4 bytes hay còn gọi là số nguyên dài (long) gồm có 2 kiểu sau:

STT	Kiểu dữ liệu	Miền giá trị (Domain)
1	long	Từ -2,147,483,648 đến 2,147,483,647
2	unsigned long	Từ 0 đến 4,294,967,295

VI.2. Kiểu số thực

Kiểu số thực là một kiểu dữ liệu mà tập các giá trị của nó là tập hợp con của tập các số thực trong toán học. Người ta định nghĩa sẵn 3 kiểu số thực với kích thước, miền giá trị và độ chính xác (số chữ số thập phân) khác nhau. Tùy thuộc vào ứng dụng của chương trình mà người lập trình lựa chọn 1 kiểu thích hợp để sử dụng.

STT	Kiểu dữ liệu	Kích thước	Miền giá trị (Domain)
1	float	4 bytes	Từ $1.2 * 10^{-38}$ đến $3.4 * 10^{38}$. Độ chính xác khoảng 7 chữ số
2	double	8 bytes	Từ $2.2 * 10^{-308}$ đến $3.4 * 10^{308}$. Độ chính xác khoảng 15 chữ số
3	long double	10 bytes	Từ $3.4 * 10^{-4932}$ đến $3.4 * 10^{4932}$. Độ chính xác khoảng 19 chữ số.

Ngoài ra ta còn có kiểu dữ liệu **void**, kiểu này mang ý nghĩa là kiểu rỗng không chứa giá trị gì cả.

VII. HẰNG

Hằng là một đại lượng mà giá trị của nó không thay đổi trong suốt quá trình thực hiện chương trình. Hằng có thể được đặt tên (theo quy tắc đặt tên trong mục IV) hoặc là một hằng trực tiếp.

Hằng có thể là một chuỗi ký tự, một ký tự, một con số xác định. Chúng có thể được biểu diễn hay định dạng (Format) với nhiều dạng thức khác nhau.

VII.2.1 Hằng số thực

Là một số thực thuộc miền giá trị của một trong 3 kiểu số thực được trình bày ở trên. Các giá trị số thực được biểu diễn theo 2 cách sau:

- *Cách 1*: Sử dụng cách viết thông thường gồm 2 phần: phần nguyên và phần thập phân; mỗi phần phân cách nhau bởi dấu chấm (.).

Thí dụ: 0.0; 123.34; -223.333; 3.00; -56.0

- *Cách 2*: Sử dụng cách viết theo số mũ hay số khoa học. Một số thực được tách làm 2 phần, cách nhau bằng ký tự e hay E

Phần giá trị: là một số nguyên hay số thực được viết theo cách 1.

Phần mũ: là một số nguyên

→ Giá trị của số thực là: *Phần giá trị nhân với 10 mũ phần mũ.*

Thí dụ: $1234.56e-3 = 1.23456$ (là số $1234.56 * 10^{-3}$)
 $-123.45E4 = -1234500$ (là $-123.45 * 10^4$)

VI.2.2 Hằng số nguyên

Một hằng số nguyên là một số nguyên thuộc miền giá trị của các số nguyên 1 byte, 2 bytes và 4 bytes.

- **Hằng số nguyên hệ thập phân:** là số nguyên có giá trị thuộc kiểu số nguyên 1 byte/2 bytes/4 bytes và được viết như cách viết thông thường trong toán học (sử dụng các ký số từ 0 đến 9 để biểu diễn một giá trị nguyên).

Thí dụ: 123 (một trăm hai mươi ba),
 -242 (trừ hai trăm bốn mươi hai).

- **Hằng số nguyên hệ bát phân:** là số nguyên có giá trị thuộc kiểu số nguyên 1 byte/2 bytes/4 bytes và được biểu diễn bởi 8 chữ số từ 0 đến 7, bắt đầu bằng số 0.

Cách biểu diễn: 0<các ký số từ 0 đến 7>

Thí dụ: 0345 (số 345 trong hệ bát phân)
 -020 (số -20 trong hệ bát phân)

Cách tính giá trị thập phân của số bát phân như sau:

Số bát phân : $0d_n d_{n-1} d_{n-2} \dots d_1 d_0$ (d_i có giá trị từ 0 đến 7)

$$\Rightarrow \text{Giá trị thập phân} = \sum_{i=0}^n d_i * 8^i$$

$$0345=229, \quad 020=16$$

• *Hằng số nguyên hệ thập lục phân*: là số nguyên có giá trị thuộc kiểu số nguyên 1 byte/2 bytes/4 bytes và được biểu diễn bởi 10 chữ số từ 0 đến 9 và 6 ký tự A, B, C, D, E, F

Ký tự	Giá trị
A	10
B	11
C	12
D	13
E	14
F	15

Cách biểu diễn:

0x<các ký số từ 0 đến 9 và 6 ký tự từ A đến F>

Thí dụ:

0x345 (số 345 trong hệ 16)

0x20 (số 20 trong hệ 16)

0x2A9 (số 2A9 trong hệ 16)

Cách tính giá trị thập phân của số thập lục phân:

Số thập lục phân : $0xd_n d_{n-1} d_{n-2} \dots d_1 d_0$

(d_i từ 0 đến 9 hoặc A đến F)

$$\Rightarrow \text{Giá trị thập phân} = \sum_{i=0}^n d_i * 16^i$$

Thí dụ: 0x345=827 , 0x20=32 , 0x2A9= 681

• *Hằng số nguyên có định trước kiểu*: Đôi khi ta muốn ghi các hằng số với kiểu được định trước trước mắt mình, điều này có thể được thực hiện bằng cách thêm một (một số) ký tự vào cuối dãy số.

- U (hoặc u): cho kiểu unsigned int

- L (hoặc l): cho kiểu long

- UL (hoặc ul): cho kiểu unsigned long

Thí dụ:

65000U: hằng số nguyên kiểu unsigned int.

123456789L: hằng số nguyên kiểu long

VI.2.3. Hằng ký tự

Hằng ký tự là một ký tự riêng biệt được viết trong cặp dấu nháy đơn (').

Thí dụ: 'a', 'A', '0', '9'

Mỗi hằng ký tự được lưu trữ đúng 1 byte trong bộ nhớ.

Một giá trị hằng ký tự là một phần tử của 1 tập hữu hạn các ký tự được sắp thứ tự. Các máy tính đều sử dụng tập các ký tự như vậy để trao đổi thông tin với nhau qua các thiết bị xuất nhập. Có nhiều cách sắp xếp bộ chữ khác nhau và có một bộ chữ (bộ mã) được sử dụng phổ biến để trao đổi thông tin giữa các thiết bị, nhất là trên máy tính. Đó là bộ mã ASCII (American Standard Code for Information Interchange).

Mỗi ký tự được mã hóa đúng bằng 1 byte, vì thế bảng mã ASCII có thể mã hóa tới 256 ký tự (2^8). Cách thức mã hóa được thực hiện bằng cách gán cho mỗi ký tự một giá trị số thuộc [0..255] gọi là mã ASCII của ký tự đó.

Thí dụ: Ký tự 'A', 'B' có mã ASCII lần lượt là 65, 66

Ký tự 'a', 'b' có mã ASCII lần lượt là 97, 98.

Thực tế chỉ có 128 giá trị số đầu tiên được sử dụng để mã hóa các ký tự thông thường, 128 giá trị tiếp theo (từ 128 - 255) được sử dụng để mã hóa cho các ký tự riêng của 1 số ngôn ngữ, các ký tự toán học...

Bảng dưới đây là bảng mã ASCII của 128 ký tự đầu tiên (còn được gọi là bảng mã ASCII chuẩn):

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NU	SOH	STX	ETX	END	ACK	BS	HT	LF	VT	FF	CR	SO	SI		
1	DL	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Chúng ta có thể thực hiện các phép toán số học trên 2 ký tự (thực chất là thực hiện phép toán trên giá trị ASCII của chúng)

VI.2.4. Hằng chuỗi ký tự

Hằng chuỗi ký tự là một chuỗi hay một xâu ký tự được đặt trong cặp dấu nháy kép (“”).

Thí dụ: “Ngon ngu lap trinh C”, “Khoa CNTT-DHCT”

Chú ý:

1. Một chuỗi không có nội dung “” được gọi là chuỗi rỗng.
2. Khi lưu trữ trong bộ nhớ, một chuỗi được kết thúc bằng ký tự NULL (“\0”: mã ASCII là 0).
3. Để biểu diễn ký tự đặc biệt bên trong chuỗi ta phải thêm dấu \ phía trước.

Thí dụ:

“I’m a student” phải viết “\I’m a student”

Một số ký tự đặc biệt

Ký tự	Giá trị thập lục phân	Ký tự được hiển thị	Ý nghĩa
\a	0x07	BEL	Phát ra tiếng chuông
\b	0x08	BS	Di chuyển con trỏ sang trái 1 ký tự và xóa ký tự bên trái (backspace)
\f	0x0C	FF	Sang trang
\n	0x0A	LF	Xuống dòng
\r	0x0D	CR	Trở về đầu dòng
\t	0x09	HT	Tab theo cột (giống gõ phím Tab)
\\	0x5C	\	Dấu \
\'	0x2C	'	Dấu nháy đơn (')
\"	0x22	"	Dấu nháy kép (")
\?	0x3F	?	Đấu chấm hỏi (?)
\0	0x00		Ký tự NULL (rỗng)

VII. BIẾN

VII.1. Biến

Biến là một đại lượng được người lập trình định nghĩa và được đặt tên thông qua việc khai báo biến. Biến dùng để lưu trữ giá trị dữ liệu trong quá trình thực hiện chương trình và giá trị của biến có thể bị thay đổi trong quá trình này. Việc đặt tên biến phải tuân theo quy tắc đặt tên trong mục IV.

Khi khai báo biến ta phải xác định kiểu cho nó. Và khi đó, giá trị của biến phải thuộc miền giá trị của kiểu.

VII.1.1. Cú pháp khai báo biến:

<Kiểu dữ liệu> Danh sách các tên biến cách nhau bởi dấu phẩy;

Thí dụ:

```
int    a, b, c; // Ba biến a, b, c có kiểu int
long n;        // Biến n có kiểu long
float   nua_chu_vi;    /*Biến nua_chu_vi
                        có kiểu float*/
double  dien_tich;     /*Biến dien_tich có
                        kiểu double*/
```

Lưu ý: Phải có dấu chấm phẩy ở cuối phần khai báo biến.

VII.1.2. Vị trí khai báo biến trong C

Trong ngôn ngữ lập trình C, ta phải khai báo biến đúng vị trí. Nếu khai báo (đặt các biến) không đúng vị trí sẽ dẫn đến những sai sót ngoài ý muốn mà người lập trình không lường trước (hiệu ứng lè). Về cơ bản, ta có 2 cách đặt vị trí của biến như sau:

a) Khai báo biến ngoài: Các biến này được đặt bên ngoài tất cả các hàm và nó có tác dụng hay ảnh hưởng đến toàn bộ chương trình (còn gọi là biến toàn cục).

Thí dụ:

```
int      i;      /*Bien ben ngoai */
float    pi;      /*Bien ben ngoai*/
main()
{ ...
}
```

b) Khai báo biến trong: Các biến được đặt ở bên trong hàm, hay trong một khối lệnh. Các biến này chỉ có tác dụng hay ảnh hưởng đến hàm hay khối lệnh chứa nó. Vì thế chúng còn được gọi là các biến cục bộ của hàm hay khối lệnh. Vị trí khai báo của các biến này là ở đầu mỗi khối lệnh.

Thí dụ 1:

```
#include <stdio.h>
#include<conio.h>
int bienngoai; /*khai bao bien ngoai*/
main()
{
    int j,i; // khai bao bien cuc bo trong ham main
    i=1; j=2;
    bienngoai=3;
    printf("\n Gia7 tri cua i la %d",i);
        /*%d : In một số nguyên*/
    printf("\n Gia tri cua j la %d",j);
    printf("\n Gia tri cua bienngoai la
        %d",bienngoai);

    getch();
}
```

Thí dụ 2:

```
#include <stdio.h>
#include<conio.h>
main ()
{ int i, j;          /*Bien cuc bo*/
  i=4; j=5;
  printf("\n Gia tri cua i la %d",i);
  printf("\n Gia tri cua j la %d",j);
  if(j>i)
  {
      int hieu=j-i; /* Bien cuc bo */
      printf("\n j tru i la %d",hieu);
  }
  else
```

```
{
    int hieu=i-j; /*Bien cuc bo*/
    printf("\n i tru j la %d",hieu);
}
getch();
}
```

VIII. BIỂU THỨC

Biểu thức là một sự kết hợp giữa các toán tử (operator) và các toán hạng (operand) theo đúng một trật tự nhất định.

Mỗi toán hạng có thể là một hằng, một biến, một hàm hoặc một biểu thức khác.

Mỗi biểu thức có một giá trị. Giá trị của 1 biểu thức có được bằng cách áp dụng toán tử lên các toán hạng.

Trong một biểu thức có nhiều toán tử có cùng độ ưu tiên thì thứ tự thực hiện các phép toán là từ trái sang phải (*quy tắc kết hợp trái*).

Trong một biểu thức có nhiều toán tử có độ ưu tiên khác nhau thì toán tử nào có độ ưu tiên cao hơn sẽ được thực hiện trước (*quy tắc ưu tiên*).

Nếu cần thay đổi thứ tự thực hiện các toán tử theo 2 quy tắc trên, ta dùng cặp dấu ngoặc đơn () để chỉ định toán tử nào sẽ được thực hiện trước.

Thí dụ: Biểu thức nghiệm của phương trình bậc hai:

$$(-b + \text{sqrt}(\text{Delta})) / (2 * a)$$

Trong đó

- 2 là hằng; a, b, Delta là biến.
- sqrt: hàm tính căn bậc 2
- +, *, /, - là các toán tử
- Biểu thức có sử dụng các dấu ngoặc đơn để tăng độ ưu tiên.

VIII.1 Các toán tử số học

Các toán tử 2 ngôi: gồm có +, -, *, /, %.

Toán tử	Miền giá trị của toán hạng	Miền giá trị của kết quả	Ý nghĩa
+	Nguyên, Thực	Nguyên, Thực	Cộng 2 số
-	Nguyên, Thực	Nguyên, Thực	Trừ 2 số
*	Nguyên, Thực	Nguyên, Thực	Nhân 2 số
/	Nguyên	Nguyên	Chia lấy phần nguyên
%	Thực	Thực	Chia 2 số, kết quả là một số thực

Tăng và giảm (++ & --)

Đây là 2 toán tử 1 ngôi để làm tăng (++) hoặc giảm (--) giá trị của biến.

Chẳng hạn:

$++x$ giống như $x = x + 1$

$x--$ giống như $x = x - 1$

Cả 2 toán tử tăng và giảm đều có thể tiền tố (đặt trước) hay hậu tố (đặt sau) toán hạng. Thí dụ: $x = x + 1$ có thể viết $x++$ (hay $++x$)

Tuy nhiên giữa tiền tố và hậu tố có sự khác biệt khi sử dụng trong 1 biểu thức.

Đối với toán tử $++$, cả 2 trường hợp đều làm tăng giá trị của biến. Nhưng $++x$ sẽ làm tăng giá trị của x lên 1 đơn vị trước khi giá trị của x được sử dụng trong khi $x++$ sẽ sử dụng giá trị của x trước, sau đó x mới tăng giá trị lên 1 đơn vị.

Thí dụ:

$x = 10$

$y = ++x$ // $y = 11$

Tuy nhiên:

$x = 10$

$y = x++$ // $y = 10$

Tương tự đối với toán tử $--$.

Thứ tự ưu tiên của các toán tử số học:

$++$ $--$ sau đó là $*$ $/$ $\%$ rồi mới đến $+$ $-$

VIII.2 Các toán tử quan hệ và các toán tử Logic

Trong C, mọi giá trị khác 0 được xem là đúng còn 0 là sai. Các biểu thức sử dụng các toán tử quan hệ và Logic gọi là các biểu thức logic và giá trị của 1 biểu thức logic là 1 hoặc 0.

Các toán tử quan hệ

Toán tử	Miền giá trị của toán hạng	Miền giá trị của kết quả	Ý nghĩa
>	Số, ký tự	0, 1	Lớn hơn
>=	Số, ký tự	0, 1	Lớn hơn hoặc bằng
<	Số, ký tự	0, 1	Nhỏ hơn
<=	Số, ký tự	0, 1	Nhỏ hơn hoặc bằng
==	Số, ký tự	0, 1	Bằng
!=	Số, ký tự	0, 1	Khác

Các toán tử logic

Toán tử	Miền giá trị của toán hạng	Miền giá trị của kết quả	Ý nghĩa
&&	0, 1	0, 1	AND
	0, 1	0, 1	OR
!	0, 1	0, 1	NOT

Bảng chân trị cho các toán tử Logic:

p	q	p&&q	p q	!p
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Các toán tử quan hệ và Logic đều có độ ưu tiên thấp hơn các toán tử số học. Do đó một biểu thức như: $10 > 1 + 12$ sẽ được xem là $10 > (1 + 12)$ và kết quả là sai (0).

Ta có thể kết hợp vài toán tử lại với nhau thành biểu thức như sau:

$10 > 5 \&\& !(10 < 9) || 3 <= 4$ Kết quả là đúng

Thứ tự ưu tiên của các toán tử quan hệ và Logic

Cao nhất: !
 > >= < <=
 == !=
 &&
 Thấp nhất: ||

VIII.3 Các toán tử thao tác bit

Đây là các toán tử cho phép thao tác trên từng bit nhị phân của các toán hạng là các số nguyên.

Toán tử	Ý nghĩa
&	AND theo từng bit
	OR theo từng bit
^	XOR theo từng bit
~	Đảo bit

Thí dụ:

- Với số nguyên 3, dạng nhị phân là:

3 = 0000 0000 0000 0011

~3 = 1111 1111 1111 1100

- 3 & 5 sẽ có kết quả:

0000 0000 0000 0011 3

0000 0000 0000 0110 6

Kết quả: 0000 0000 0000 0010 4

Bảng chân trị của toán tử ^ (XOR)

p	q	p^q
0	0	0
0	1	1
1	0	1
1	1	0

Phép toán dịch trái và dịch phải:

Phép tính nhân một số nguyên với một số là lũy thừa của 2 có thể thực hiện nhanh hơn nếu dùng $>>$ (dịch phải) hoặc $<<$ (dịch trái).

- $N << M$: dịch sang trái số nguyên N đi M bit, tương đương $N * 2^M$.

- $N >> M$: dịch sang phải số nguyên N đi M bit, tương đương $N / 2^M$.

Thí dụ: $4 << 3$, kết quả là 32 ($4 * 2^3$).

VIII.4 Toán tử ? cùng với :

C có một toán tử rất mạnh và trong 1 số trường hợp có thể dùng để thay thế cho các câu lệnh của If-Then-Else. Cú pháp của việc sử dụng toán tử ? là:

$E1 \quad ? \quad E2 \quad : \quad E3$

Trong đó $E1$, $E2$, $E3$ là các biểu thức.

Ý nghĩa: Trước tiên $E1$ được ước lượng, nếu đúng $E2$ được ước lượng và nó trở thành giá trị của biểu thức; nếu $E1$ sai, $E2$ được ước lượng và trở thành giá trị của biểu thức.

Thí dụ:

$X = 10$

$Y = X > 9 ? 100 : 200$

Thì Y được gán giá trị 100, nếu X nhỏ hơn 9 thì Y sẽ nhận giá trị là 200. Đoạn mã này tương đương cấu trúc if như sau:

$X = 10$

if ($X < 9$) $Y = 100$

else $Y = 200$

VIII.5 Toán tử con trỏ & và *

Một con trỏ là địa chỉ trong bộ nhớ của một biến. Một biến con trỏ là một biến được khai báo riêng để chứa một con trỏ đến một đối tượng của kiểu đã chỉ ra nó. Ta sẽ tìm hiểu kỹ hơn về con

trở trong chương về con trỏ. Phần này chỉ đề cập ngắn gọn đến hai toán tử thường được sử dụng để thao tác với các con trỏ.

Toán tử thứ nhất là `&`, là một toán tử trả về địa chỉ bộ nhớ của 1 biến.

Thí dụ:

`m = &count`

Đặt vào biến `m` địa chỉ bộ nhớ của biến `count`.

Chẳng hạn, biến `count` ở vị trí bộ nhớ 2000, giả sử `count` có giá trị là 100. Sau câu lệnh trên `m` sẽ nhận giá trị 2000.

Toán tử thứ hai là `*`, là một bổ sung cho `&`; đây là một toán tử trả về giá trị của biến được cấp phát tại địa chỉ theo sau đó.

Thí dụ:

`q = *m`

Sẽ đặt giá trị của `count` vào `q`. Bây giờ `q` sẽ có giá trị là 100 vì 100 được lưu trữ tại địa chỉ 2000.

VIII.6 Toán tử dấu phẩy ,

Toán tử dấu , được sử dụng để kết hợp các biểu thức lại với nhau. Bên trái của toán tử dấu , luôn được xem là kiểu void. Điều đó có nghĩa là biểu thức bên phải trở thành giá trị của tổng các biểu thức được phân cách bởi dấu phẩy.

Thí dụ: `x = (y=3, y+1);`

Trước hết gán 3 cho `y` rồi gán 4 cho `x`. Cặp dấu ngoặc đơn là cần thiết vì toán tử dấu , có độ ưu tiên thấp hơn toán tử gán.

VIII.7 Xem các dấu ngoặc đơn và cặp dấu ngoặc vuông là toán tử

Trong C, cặp dấu ngoặc đơn là toán tử và chúng được dùng để tăng độ ưu tiên của các biểu thức bên trong nó.

Các cặp dấu ngoặc vuông thực hiện thao tác truy xuất phần tử trong mảng (sẽ được trình bày rõ ràng hơn trong chương Mảng).

VIII.8 Tổng kết về độ ưu tiên

Cao nhất	() []
	! ~ ++ -- (Kiểu) * &
	* / %
	+ -
	<< >>
	< <= > >=
	&
	^
	&&
	?:
	= += -= *= /=
Thấp nhất	,

VIII.9 Phép gán mở rộng trong C

Trong một số trường hợp, ta có thể viết tắt như sau: $x += 10$. Thực chất của cách viết này là tương đương với cách viết $x = x + 10$.

Cách viết như trong thí dụ vừa nêu có thể thực hiện trên tất cả các toán tử hai ngôi của C. Tổng quát:

$$(Biến) = (Biến) (Toán tử) (Biểu thức)$$

có thể được viết:

$$(Biến) (Toán tử) = Biểu thức$$

IX. CẤU TRÚC CỦA 1 CHƯƠNG TRÌNH C

IX.1. Tiền xử lý và biên dịch

Trong C, việc dịch một tập tin nguồn được tiến hành trên hai bước hoàn toàn độc lập với nhau:

- Tiền xử lý.
- Biên dịch.

Hai bước này trong phần lớn thời gian được nối tiếp với nhau một cách tự động theo cách thức mà ta có ấn tượng rằng nó đã được thực hiện như là một xử lý duy nhất. Nói chung, ta thường nói đến việc tồn tại của một bộ tiền xử lý (preprocessor) nhằm chỉ rõ chương trình thực hiện việc xử lý trước. Ngược lại, các thuật ngữ trình biên dịch hay sự biên dịch vẫn còn nhập nhằng bởi vì nó chỉ ra khi thì toàn bộ hai giai đoạn, khi thì lại là giai đoạn thứ hai.

Bước tiền xử lý tương ứng với việc cập nhật trong văn bản của chương trình nguồn, chủ yếu dựa trên việc diễn giải các mã lệnh rất đặc biệt gọi là các chỉ thị dẫn hướng của bộ tiền xử lý (destination directive of preprocessor); các chỉ thị này được nhận biết bởi chúng bắt đầu bằng ký hiệu (symbol) #.

Hai chỉ thị quan trọng nhất là:

- Chỉ thị sự gộp vào các tập tin nguồn khác: `#include`
- Chỉ thị việc định nghĩa các macros hoặc ký hiệu: `#define`

Chỉ thị đầu tiên được sử dụng trước hết là nhằm gộp vào nội dung của các tập tin cần có (tập tin thư viện). Chỉ thị này thường sử dụng vì bởi các hàm của thư viện chuẩn của C được định nghĩa trong các tập tin thư viện; do đó, muốn sử dụng các hàm này, tập tin thư viện định nghĩa chúng phải được gộp vào.

Thí dụ: `#include <stdio.h>`

Chỉ thị thứ hai rất hay được sử dụng trong các tập tin thư viện (header file) đã được định nghĩa trước đó và thường được khai thác bởi các lập trình viên trong việc định nghĩa các ký hiệu như là:

```
#define NB_COUPS_MAX 100  
#define SIZE 25
```

IX.2 Cấu trúc một chương trình C

Một chương trình C bao gồm các phần như: Các chỉ thị tiền xử lý, khai báo biến ngoài, các hàm tự tạo, chương trình chính (hàm main).

Cấu trúc có thể như sau:

Các chỉ thị tiền xử lý (Preprocessor directives)

```
#include <Tên tập tin thư viện>
```

```
#define ....
```

Thí dụ:

```
#include<stdio.h>
```

```
#define MAXINT 32767
```

Định nghĩa kiểu dữ liệu (phần này không bắt buộc): dùng để đặt tên lại cho một kiểu dữ liệu nào đó để gọi nhớ hay đặt 1 kiểu dữ liệu cho riêng mình dựa trên các kiểu dữ liệu đã có.

Cú pháp: `typedef <Tên kiểu cũ> <Tên kiểu mới>;`

Thí dụ: `typedef int SoNguyen; // Kiểu SoNguyen là kiểu int`

Khai báo các prototype (tên hàm, các tham số, kiểu kết quả trả về,... của các hàm sẽ cài đặt trong phần sau, phần này không bắt buộc): phần này chỉ là các khai báo đầu hàm, không phải là phần định nghĩa hàm.

Khai báo các biến ngoài (các biến toàn cục) *phần này không bắt buộc*: phần này khai báo các biến toàn cục được sử dụng trong cả chương trình.

Chương trình chính (hàm main), phần này bắt buộc phải có `main()`

```
{
```

Các khai báo cục bộ trong hàm main: Các khai báo này chỉ tồn tại trong hàm mà thôi, có thể là khai báo biến hay khai báo kiểu.

Các câu lệnh dùng để định nghĩa hàm main

```
}
```

Cài đặt các hàm

```
<Kiểu dữ liệu trả về> <Tên hàm> (các tham số)
{
    Các khai báo cục bộ trong hàm.
    Các câu lệnh dùng để định nghĩa hàm
    [return <kết quả trả về>;]
}
...
```

Một chương trình C được thực thi từ hàm main (thông thường là từ câu lệnh đầu tiên đến câu lệnh cuối cùng).

IX.3 Cú pháp khai báo các phần bên trong một chương trình C

IX.3.1. Chỉ thị *#include* để sử dụng tập tin thư viện

Cú pháp:

```
#include <Tên tập tin>
hay #include "Tên đường dẫn"
```

Thí dụ: #include <stdio.h>

Nếu ta dùng *#include* "Tên đường dẫn" thì ta phải chỉ rõ tên ở đâu, tên thư mục và tập tin thư viện.

Thí dụ: #include "C:\\TC\\math.h"

Trong trường hợp tập tin thư viện nằm trong thư mục hiện hành thì ta chỉ cần đưa tên tập tin thư viện. Thí dụ: *#include* "math.h".

IX.3.2. Chỉ thị *#define* để định nghĩa một tên

Chỉ thị này có cú pháp dạng đơn giản như sau:

```
#define <Tên> <Giá trị>
```

Ví dụ:

```
#define MAXINT 32767
```

Trong thí dụ trên, chỉ thị define định nghĩa một tên MAXINT; tên này có thể sử dụng như một hằng nguyên có giá trị là 32767.

IX.3.3 Khai báo các prototype của hàm

Cú pháp:

<Kiểu kết quả trả về> Tên hàm (danh sách đối số)

Thí dụ:

```
long giaithua( int  n);  
double  x_mu_y(float  x, float y);
```

IX.3.4. Cấu trúc của hàm main

Hàm main chính là chương trình chính, gồm các khai báo, các lệnh xử lý, các lời gọi các hàm khác.

Cú pháp:

main(đối số)

{

Các khai báo và các câu lệnh định nghĩa hàm

}

Thí dụ:

```
main()  
{  
    int a=5, b=6,c;  
    float  x=3.5, y=4.5,z;  
    printf("Day la chuong trinh chinh");  
    c= a + b;  
    printf("\n%d + %d = %d",a,b,c);  
    z= x+y;  
    printf("\n%f + %f = %f", x,y,z);  
    getch();  
}
```